

Write a Java program with a superclass Plant that has a method needsWater() which returns a boolean value. A subclass Cactus should override this method to return false, signifying that cacti require less water

```
class Plant {  
    // This method returns a boolean value indicating whether the plant needs water  
    public boolean needsWater() {  
        return true; // Most plants need water  
    }  
}  
class Cactus extends Plant {  
    // Override the needsWater method for the Cactus subclass  
    @Override  
    public boolean needsWater() {  
        return false; // Cacti require less water  
    }  
}  
public class PlantDemo {  
    public static void main(String[] args) {  
        Plant myPlant = new Plant();  
        Cactus myCactus = new Cactus();  
  
        // Check if the plants need water  
        System.out.println("Does the plant need water? " + myPlant.needsWater());  
        System.out.println("Does the cactus need water? " + myCactus.needsWater());  
    }  
}
```

```
import java.util.Scanner;
class Plant {
    // This method returns a boolean value indicating whether the plant needs water
    public boolean needsWater() {
        return true; // Most plants need water
    }
}
class Cactus extends Plant {
    // Override the needsWater method for the Cactus subclass
    @Override
    public boolean needsWater() {
        return false; // Cacti require less water
    }
}
public class PlantDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter the type of plant ('Plant' or 'Cactus'):");
        String type = scanner.nextLine();
        Plant myPlant;
        if(type.equalsIgnoreCase("Cactus")) {
            myPlant = new Cactus();
        } else {
            myPlant = new Plant();
        }
        // Check if the plant needs water based on the type
        System.out.println("Does the " + type + " need water? " + myPlant.needsWater());
    }
}
```

Create a Java program with three levels of inheritance. At the top, have a class Instrument with a method play(). Below it, have a class StringInstrument that extends Instrument. Finally, have a class Guitar that extends StringInstrument and overrides the play() method to print "Guitar is playing chords".

```
// Top-level class in the inheritance hierarchy
class Instrument {
    // Method that can be overridden by subclasses
    public void play() {
        System.out.println("Instrument is playing");
    }
}
// Second level in the hierarchy, extending the Instrument class
class StringInstrument extends Instrument {
    // Inherits the play() method from Instrument
}
// Third level in the hierarchy, extending the StringInstrument class
class Guitar extends StringInstrument {
    // Overrides the play() method to provide specific implementation for Guitar
    @Override
    public void play() {
        System.out.println("Guitar is playing chords");
    }
}
// Main class to run the program
public class MusicDemo {
    public static void main(String[] args) {
        // Create an instance of Guitar, which is a StringInstrument, which is also an Instrument
        Guitar myGuitar = new Guitar();
        // Call the play method on the Guitar instance
        myGuitar.play();
    }
}
```

```
import java.util.Scanner;
// Top-level class in the inheritance hierarchy
class Instrument {
    // Method that can be overridden by subclasses
    public void play() {
        System.out.println("Instrument is playing");
    }
}
// Second level in the hierarchy, extending the Instrument class
class StringInstrument extends Instrument {
    // Inherits the play() method from Instrument
}
// Third level in the hierarchy, extending the StringInstrument class
class Guitar extends StringInstrument {
    // Overrides the play() method to provide specific implementation for Guitar
    @Override
    public void play() {
        System.out.println("Guitar is playing chords");
    }
}
// Main class to run the program
public class MusicDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Prompt the user for the type of instrument to play
        System.out.println("Enter the type of instrument to play (Instrument, StringInstrument, Guitar):");
        String instrumentType = scanner.nextLine();
        Instrument instrument;
        // Determine the type of instrument based on user input
        switch (instrumentType) {
            case "Guitar":
                instrument = new Guitar();
                break;
            case "StringInstrument":
                instrument = new StringInstrument();
                break;
            default:
                instrument = new Instrument();
                break;
        }
        // Call the play method on the created instance
        instrument.play();
    }
}
```

Super Keyword

Write a Java program that demonstrates the use of the super keyword to access a superclass's constructor and methods. Create a superclass Food with a property taste and a method printTaste(). Then, create a subclass Fruit that uses super to set the taste and to invoke the printTaste() method of Food.

```
// Superclass
class Food {
    protected String taste;
    // Constructor
    public Food(String taste) {
        this.taste = taste;
    }
    // Method to print taste
    public void printTaste() {
        System.out.println("Taste: " + taste);
    }
}

// Subclass
class Fruit extends Food {
    // Constructor
    public Fruit(String taste) {
        // Calling superclass constructor using super keyword
        super(taste);
    }
    // Method to demonstrate invoking superclass method using super keyword
    public void describeTaste() {
        System.out.print("Fruit ");
        super.printTaste(); // Invoking superclass method using super keyword
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        // Creating object of subclass Fruit
        Fruit apple = new Fruit("Sweet");
        // Invoking subclass method which in turn invokes superclass method
        apple.describeTaste();
    }
}
```


Basic Inheritance

Write a Java program where class Vehicle is the superclass with a method displayMaxSpeed() that prints out a default speed. Class Car extends Vehicle and overrides the displayMaxSpeed() method to print out a higher speed.

Method Overriding

Write a Java program with a superclass Plant that has a method needsWater() which returns a boolean value. A subclass Cactus should override this method to return false, signifying that cacti require less water.

Multi-level Inheritance

Create a Java program with three levels of inheritance. At the top, have a class Instrument with a method play(). Below it, have a class StringInstrument that extends Instrument. Finally, have a class Guitar that extends StringInstrument and overrides the play() method to print "Guitar is playing chords".

Super Keyword

Write a Java program that demonstrates the use of the super keyword to access a superclass's constructor and methods. Create a superclass Food with a property taste and a method printTaste(). Then, create a subclass Fruit that uses super to set the taste and to invoke the printTaste() method of Food.